Serial multiplier

Firstly, I designed the full serial multiplier in RTL abstraction for ease before breaking it down to structural and RTL.

I then made a structural design which is not needed for testing however if there was any reason in a bigger project then you could always use the structural design for it.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity serial_multiplier is
    port (
        clk     : in  std_logic;
        reset   : in  std_logic;
        start   : in  std_logic;    --Start the multiplication
        a       : in  std_logic_vector(3 downto 0);  -- 1st Input (4 bits)
        b       : in  std_logic_vector(3 downto 0);  -- 2nd Input (4 bits)
        done    : out std_logic;    --End the multiplication
        product : out std_logic_vector(7 downto 0) --Result of multiplication as an 8 bit integer.
    );
end entity;

architecture rtl of serial_multiplier is
    signal A_reg : std_logic_vector(3 downto 0) := (others => '0'); --Storing the current value of A
    signal B_reg : std_logic_vector(3 downto 0) := (others => '0'); --Storing fixed value of B (As this stays constant in the add and shift method)
    signal P_reg : std_logic_vector(7 downto 0) := (others => '0'); --Stores the partial product value (By the end it will be the product)
    signal cnt   : integer range 0 to 4 := 0; --Shift counter
    signal busy  : std_logic := '0'; --sets busy to 0
begin
    process(clk, reset) --Only works on rising clock edges or when there is a reset
    begin
        if reset = '1' then --Setting all register values to 0 as well as busy and shift counter
            A_reg   <= (others => '0'); --Set all bits of A register to 0
            B_reg   <= (others => '0');
            P_reg   <= (others => '0');
            cnt     <= 0;
            busy    <= '0';
            done    <= '0';

        elsif rising_edge(clk) then
            if busy = '0' then
                done <= '0';
                if start = '1' then
                    A_reg <= a; --Initialise the register to the input a (Set from testbench)
                    B_reg <= b; --Initialise the register to the input b (Set from testbench)
                    P_reg <= (others => '0');   --Setting initial partial product to 0 (PP should be 0 for this method initially)
                    cnt   <= 0;
                    busy  <= '1';   --Multiplication now in process
                end if;
            else
                if A_reg(0) = '1' then  --If current bit is 1
                    P_reg <= std_logic_vector(unsigned(P_reg) +
                             (resize(unsigned(B_reg), 8) sll cnt));  --PP added to the now 8 bit value of b(0 extended) with a left shift equal to cnt number
                end if;

                A_reg <= '0' & A_reg(3 downto 1);   --Shift A to the right (filling the left bit with a 0)
                cnt <= cnt + 1; --Add 1 to clock counter

                if cnt = 3 then --When clock = 3 then stop the loop
                    busy <= '0';
                    done <= '1';
                end if;
            end if;
        end if;
    end process;

    product <= P_reg;
end architecture;
```

Before simulating we can presume that there would be longer delays with the serial multiplier because each bit is added individually. It then must shift and add again until first partial product is obtained and repeats 4 times for all partial products that are added incrementally.

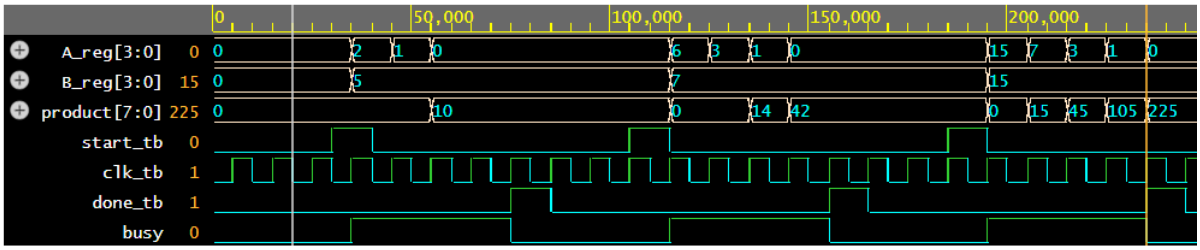

Figure 1: Simulation results

This simulation shows how the serial multiplier acts on a clock cycle. When start is high, this is when the calculations start. As soon as the input values are generated, each rising edge shifts a to the right and checks if the LSB is a 0 or 1. If it's 0 then skip addition but if it is a 1 then add b shifted to the left to the product. The product in figure 1 shows the stages with the different partial products. Done is always high after 4 iterations since this is a 4 x 4 multiplier. Busy is high between the done highs to show that it is doing calculation when running. We can see from simulation that the full calculation took 215 ns.

https://www.edaplayground.com/x/AjPH

Question 3-

The simulation was 20ns for parallel multiplication and 215ns for series multiplication. We can see that the parallel multiplication can be done much faster however the trade off is that it needs more space as it uses more full adders whilst the series multiplication reuses the same full adder. There is less delay but also takes up more space and with more components, makes it more expensive.